

The Clerk's Arithmetic Manual

How the clerk in the office does math on numbers too big for its head.
Everything below is one idea repeated: ****a giant number written as four smaller chunks, and school rules applied chunk by chunk.****

0. The worksheet

Every sheet of paper on the clerk's desk holds a 256-bit number. The clerk's brain holds 64 bits at a time, so each number is written as FOUR chunks of 64 bits:

l[3]	l[2]	l[1]	l[0]
+-----+	+-----+	+-----+	+-----+
big end			small end
+-----+	+-----+	+-----+	+-----+

value = l[3]*B³ + l[2]*B² + l[1]*B + l[0] where B = 2⁶⁴

Each chunk is a "digit" in base 2⁶⁴. Just like 4702 means 4*10³ + 7*10² + 0*10 + 2, except our digits go up to ~1.8 * 10¹⁹.

THE ONE RULE OF THE ROOM: the page is a ring. Arithmetic wraps mod 2²⁵⁶. Anything that falls off the left edge is gone. MAX + 1 = 0.

1. Comparing (< <= > >=)

Idea: most significant chunk wins. Scan from l[3] down; the first chunk that differs decides. Like comparing 4700 vs 4699 - the hundreds digit settles it, you never look at the rest.

a:	[0]	[0]	[5]	[9]
b:	[0]	[0]	[7]	[1]
			^	
l[3]:	0 == 0			tie
l[2]:	0 == 0			tie
l[1]:	5 < 7			DECIDED -> a < b (l[0] never consulted)

Walkthrough: compare U(5,0,1,0) vs U(5,0,2,0) -> l[3], l[2] tie, l[1] differs, 1 < 2 -> a < b. Done.

The family trick: write < once. > is b < a. <= is !(b < a). >= is !(a < b). One real algorithm, three one-liners.

2. Adding (+)

Idea: column addition. Add chunk by chunk starting at l[0]; when a column overflows past 64 bits, carry a 1 into the next column.

The trick for detecting a carry: an unsigned add that overflows WRAPS,

so the result comes out smaller than what you added:

```
s = x + y;    carry happened iff    s < x    (sum wrapped past 2^64)
```

Walkthrough: $U(M, 0, 0, 0) + 1$ (M = all-ones chunk)

```
l[0]:  M + 1 = 0    (wrapped!)    carry = 1    because 0 < M
l[1]:  0 + 0 + 1 = 1            carry = 0
l[2], l[3]: 0
-> U(0, 1, 0, 0) = 2^64    [checkmark]
```

The carry out of $l[3]$ falls off the page -- that IS mod 2^{256} .

3. Subtracting (-)

Idea: same dance, borrows instead of carries. $a - b$ underflows a chunk when $a < b$ -- and wrapping means the machine borrows $+2^{64}$ for free (you just owe the next chunk 1).

```
s = a.l[i] - borrow;    // paying last column's debt
c = (a.l[i] < borrow);  // did paying the debt underflow?
c |= (s < b.l[i]);      // did subtracting b underflow?
s -= b.l[i];
borrow = c;             // pass the debt upward
```

Walkthrough: $U(0, 1, 0, 0) - 1$ ($2^{64} - 1$)

```
l[0]:  0 - 0 - 1 -> wraps to M, borrow = 1
l[1]:  1 - 1 - 0 = 0, borrow = 0
-> U(M, 0, 0, 0)    [checkmark]
```

The final borrow off $l[3]$ is FORGIVEN -- that's why $0 - 1 = \text{MAX}$.

Negatives live at the top of the ring: $-1 = \text{MAX}$, $-2 = \text{MAX} - 1$, and the top bit (bit 255) is the sign flag.

4. Bitwise (& | ^ ~)

The lazy ops. Chunks don't talk to each other at all -- bit k of the answer depends only on bit k of a and b .

```
r.l[i] = a.l[i] & b.l[i];    // four independent ANDs. done.
```

5. Shifting (<< >>)

Idea: shift $n = (n/64 \text{ whole chunks}) + (n\%64 \text{ leftover bits})$. Chunks slide whole; leftover bits spill ACROSS chunk boundaries through a doorway we build by hand.

```
<< 3:    l[3] <- l[2] <- l[1] <- l[0] <- 0
          each source also donates its top bits to the limb above
```

```

<< 74 = << (64 + 10):
    limb_shift = 1, bit_shift = 10
    dest i reads  a.l[i-1] << 10   |   a.l[i-2] >> (64-10)
                  |               |
                  slid up         the 10 bits that fell off
                  one limb        the limb below's top edge

```

Walkthrough: $U(1) \ll 64 \rightarrow$ limb_shift 1, bit_shift 0 $\rightarrow$ l[1] gets l[0]'s content whole $\rightarrow U(0, 1, 0, 0) = 2^{64}$.

THE TRAP: when bit_shift == 0 the spill term computes $\gg 64$, and shifting a 64-bit value by 64 is UNDEFINED BEHAVIOR (the CPU masks the count to 6 bits $\rightarrow \gg 64$ silently acts like $\gg 0$). Always guard with `if (bit_shift && ...)`.

Bits that slide off l[3]'s top ($\ll$) or l[0]'s bottom ($\gg$) fall off the page. Gone.

6. Bit / byte access (bit, byte_at, bit_length)

The indexing idiom used everywhere: bit i lives at

```

limb      = i / 64    (i >> 6)
position  = i % 64    (i & 63)

bit(a, i) = (a.l[i/64] >> (i%64)) & 1    // 0 = least significant
set bit:  q.l[i/64] |= (1 << (i%64))

```

byte_at flips convention on purpose: byte 0 = MOST significant byte, because the BYTE opcode reads the worksheet like a human reads the number (big-endian). `byte_at(v, 31)` is the last byte of l[0].

bit_length = index of the highest set bit + 1; 0 for zero.

7. Multiplying (*)

The grid. Each chunk is a digit; a digit x digit product needs TWO digit slots (like $7 \times 8 = 56$). `_umul128` gives you the two halves: lo and hi.

```

      a3 a2 a1 a0
    x b3 b2 b1 b0
    -----
every (i, j) pair:  ai * bj = hi:B + lo
                   lo lands in column i+j
                   hi lands in column i+j+1    (the index IS the
                                                place value)

```

Walkthrough: $U(0,1,0,0) * U(0,1,0,0)$ ($2^{64} * 2^{64}$)

```

only nonzero pair: i=1, j=1 -> column i+j = 2
_umul128(1, 1) -> lo=1, hi=0
r.l[2] += 1
-> U(0, 0, 1, 0) = 2^128    [checkmark]

```

Carry bookkeeping: c is a FLAG (0/1) for "did this add wrap"; carry is a VALUE (hi can be a whole limb). hi $\leq$ B-2 because the biggest digit product is $(B-1)^2 = (B-2)B + 1$ -- like $9 \times 9 = 81$ never carries a 10.

8. Dividing (/ %)

Binary long division: the quotient digit can only be 0 or 1, so there's no guessing -- just "does the divisor fit?"

```
q = 0, r = 0
for i = 255 down to 0:           // peel a's bits, MSB first
    r = (r << 1) | bit(a, i)     // "bring down the next digit"
    if r >= b:                   // divisor fits?
        r = r - b; set bit i of q
```

Walkthrough: $13 / 3$ ($a = 1101$, $b = 0011$)

```
i=3: r = 0<<1 | 1 = 1    1 >= 3? no      q = 0000
i=2: r = 1<<1 | 1 = 3    3 >= 3? yes    r = 0    q = 0100
i=1: r = 0<<1 | 0 = 0    0 >= 3? no      q = 0100
i=0: r = 0<<1 | 1 = 1    1 >= 3? no      q = 0100
-> q = 4, r = 1    and 13 = 4*3 + 1 [checkmark]
```

Rulebook quirks: $x / 0 = 0$, $x \% 0 = 0$ -- no panicking allowed.

9. Signed ops (slt sgt sdiv smod)

Same ink, different reading. The bits don't change; bit 255 = sign.

unsigned: 0xFF..FF = $2^{256} - 1$	signed: 0xFF..FF = -1
unsigned: 0x80..00 = 2^{255}	signed: 0x80..00 = -2^{255}

slt: compare as signed -- if signs differ, negative loses; if same sign, compare normally (but for two negatives, bigger magnitude = smaller).

sdiv/smod: work on magnitudes, fix the sign after -- result takes the sign of the dividend; sdiv(-2^{255} , -1) wraps back to -2^{255} .

10. addmod / mulmod -- the trap

$(a + b) \bmod n$ CANNOT be done as "add, wrap, then mod" -- because $a + b$ itself can overflow the page, and the wrapped residue is a DIFFERENT number mod n .

addmod(max, max, 3):		
naive:	(max + max) wraps to $2^{256} - 2$, % 3 = 2	WRONG
exact:	$2^{257} - 2 \bmod 3 = 0$	RIGHT

The clerk needs a hidden wider worksheet (a 257-bit or 512-bit intermediate) that never goes on the stack. mulmod needs the full 512 bits of $a*b$ before reducing mod n .

11. Conversions (from_bytes, to_bytes32, hex io)

The endianness boundary. Inside: limbs are little-endian (l[0] small).

On the wire/paper: big-endian (most significant byte first). Every conversion is just walking the right direction:

```

from_bytes:  p[0] is the BIG end -> lands in the TOP of l[3]
              p[n-1] is small end -> lands in the BOTTOM of l[0]
to_bytes32:  reverse the walk
from_hex:    parse digits, shift-and-add (or nibble into limbs)
to_hex:      nibble-walk limbs top-down, skip leading zeros

```

Rules of the room (summary)

- The page is a ring: everything wraps mod 2^{256} .
- Carries/borrows between chunks are real; off the page edge, dropped.
- Each chunk is a digit in base 2^{64} . Place value = index.
- Products are two digits wide; quotient digits are only 0 or 1.
- The clerk never panics: $x/0 = 0$, and halting is a result.
- Signed vs unsigned is a READING, not a different number.